MLA0613 - Fundamentals of Machine Learning

NAME - Y Sanjeeva Harshith Reddy
REG - 1911260107
COURSE- MACHINE LEARNING
CODE - ITA0613

1. Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Problem 1: EnjoySport Concept Learning

1. Question

Implement and demonstrate the FIND-S algorithm to find the most specific hypothesis for the EnjoySport concept using the given training dataset.

2. Aim

To implement the FIND-S algorithm in Python and find the most specific hypothesis that describes the conditions under which a person enjoys a sport.

2. Algorithm

Start the program.

Initialize the hypothesis with the first positive training example.

Read each training example.

If the example is positive, compare its attributes with the current hypothesis.

If an attribute differs, replace it with ?.

Ignore negative examples.

Display the final hypothesis

Stop the program.

4. Code

```
data = [   ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],   ["Sunny",
"Warm", "High", "Strong", "Warm", "Same", "Yes"],   ["Rainy", "Cold", "High", "Strong",
"Warm", "Change", "No"],   ["Sunny", "Warm", "High", "Strong", "Cool", "Change",
"Yes"]]hypothesis = Nonefor row in data:   if row[-1] == "Yes":      if hypothesis is None:
hypothesis = row[:-1].copy()
else:
for i in range(len(hypothesis)):
if hypothesis[i] != row[i]:
hypothesis[i] ="?"print("Final Hypothesis:", hypothesis)
```

5. Input

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

6. Output

Final Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']

7. Result

Thus, the FIND-S algorithm was implemented successfully, and the most specific hypothesis for the EnjoySport dataset was obtained.

Problem 2: Student Placement Prediction

1. Question

Implement the FIND-S algorithm to identify the most specific hypothesis for predicting whether a student gets placed based on academic performance and skills.

2. Aim

To find the most specific hypothesis for student placement using positive training examples.

3. Algorithm

Initialize an empty hypothesis.

Read the student training dataset.

Select the first example with Placement = Yes.

Compare the attributes of subsequent positive examples.

Replace mismatched attributes with ?.

Ignore negative examples.

Print the final hypothesis.

4. Code

```
data = [    ["High","Good","Yes","Good","Yes"],    ["High","Good","No","Good","Yes"],
```

```
["Medium","Average","Yes","Good","No"],   ["High","Good","Yes","Average","Yes"]]
hypothesis = Nonefor row in data:
if row[-1] == "Yes":
if hypothesis is None:
hypothesis = row[:-1].copy()
else:         f
or i in range(len(hypothesis)):
if hypothesis[i] != row[i]:
hypothesis[i] ="?"print("Final Hypothesis:", hypothesis)
```

5. Input

Academic Performance	Communication	Internship	Technical Skills	Placed
High	Good	Yes	Good	Yes
High	Good	No	Good	Yes
Medium	Average	Yes	Good	No
High	Good	Yes	Average	Yes

6. Output

Final Hypothesis: ['High', 'Good', '?', '?']

7. Result

Thus, the FIND-S algorithm identified the most specific hypothesis for student placement prediction as ['High', 'Good', '?', '?'].

Problem 3: Weather-Based Picnic Planning

1. Question

Implement the FIND-S algorithm to find the most specific hypothesis for deciding whether weather conditions are suitable for a picnic.

2. Aim

To determine the weather conditions associated with a suitable picnic using the FIND-S algorithm.

3. Algorithm

Start the program.

Initialize the hypothesis as empty.

Read all training examples.

Consider only positive examples where Picnic = Yes

Initialize the hypothesis using the first positive example.

Generalize attributes that differ by replacing them with ?.

Display the final hypothesis.

4. Code

```
data = [
["Sunny","Warm","Low","Yes"],   ["Sunny","Mild","Low","Yes"],
["Cloudy","Cold","High","No"],   ["Sunny","Warm","High","Yes"],
["Rainy","Cold","High","No"]]
hypothesis = Nonefor row in data:
if row[-1] == "Yes":
if hypothesis is None:
hypothesis = row[:-1].copy()
else:
for i in range(len(hypothesis)):
if hypothesis[i] != row[i]:
hypothesis[i] ="?"print("Final Hypothesis:", hypothesis)
```

5. Input

Weather	Temperature	Humidity	Picnic
Sunny	Warm	Low	Yes
Sunny	Mild	Low	Yes
Cloudy	Cold	High	No
Sunny	Warm	High	Yes
Rainy	Cold	High	No

6. Output

Final Hypothesis: ['Sunny', '?', '?']

7. Result

Thus, the FIND-S algorithm successfully identified the most specific hypothesis for picnic planning as ['Sunny', '?', '?'].

Problem 4: Laptop Purchase Preference

1. Question

Implement the FIND-S algorithm to find the most specific hypothesis for identifying laptop features preferred by a customer.

2. Aim

To determine the laptop characteristics associated with a customer's purchase preference using the FIND-S algorithm.

3. Algorithm

Read the laptop preference dataset.

Initialize the hypothesis with the first positive example.

Examine each training example.

Ignore examples where Purchase = No.

Compare each positive example with the hypothesis.

Generalize differing attributes using ?

Print the final hypothesis.

4. Code

```
data = [
["Dell","16GB","512GB","High","Yes"],   ["HP","16GB","512GB","High","Yes"],
["Lenovo","8GB","256GB","Medium","No"],   ["Dell","16GB","1TB","High","Yes"],
["Apple","8GB","256GB","Medium","No"]]
hypothesis = Nonefor row in data:
if row[-1] == "Yes":
if hypothesis is None:
hypothesis = row[:-1].copy()
else:
for i in range(len(hypothesis)):
if hypothesis[i] != row[i]:
hypothesis[i] ="?"print("Final Hypothesis:", hypothesis)
```

5. Input

Brand	RAM	Storage	Price Range	Purchase
Dell	16GB	512GB	High	Yes
HP	16GB	512GB	High	Yes
Lenovo	8GB	256GB	Medium	No
Dell	16GB	1TB	High	Yes
Apple	8GB	256GB	Medium	No

6. Output

Final Hypothesis: ['?', '16GB', '?', 'High']

7. Result

Thus, the FIND-S algorithm successfully obtained the most specific hypothesis for the given laptop purchase preference dataset.

Problem 5: Crop Growth Suitability

1. Question

Implement the FIND-S algorithm to determine the most specific hypothesis for identifying suitable conditions for crop growth.

2. Aim

To identify the environmental conditions associated with successful crop growth using the FIND-S algorithm.

3. Algorithm

Start the program.

Read the crop growth dataset.

Initialize the hypothesis with the first positive example.

Process all remaining training examples.

Ignore negative examples.

Compare the attributes of each positive example

Replace differing attributes with ?.

Display the final hypothesis

4. Code

```
data = [
["Black","High","Moderate","Good","Yes"],  ["Black","High","Low","Good","Yes"],
["Red","Low","Low","Poor","No"],  ["Black","High","Moderate","Average","Yes"],
["Red","Low","Moderate","Poor","No"]]
hypothesis = Nonefor row in data:
if row[-1] == "Yes":
if hypothesis is None:
hypothesis = row[:-1].copy()
else:
for i in range(len(hypothesis)):
if hypothesis[i] != row[i]:
hypothesis[i] ="?"print("Final Hypothesis:", hypothesis)
```

5. Input

Soil Type	Rainfall	Temperature	Fertility	Crop Growth
Black	High	Moderate	Good	Yes
Black	High	Low	Good	Yes
Red	Low	Low	Poor	No
Black	High	Moderate	Average	Yes
Red	Low	Moderate	Poor	No

6. Output

Final Hypothesis: ['Black', 'High', '?', '?']

7. Result

Thus, the FIND-S algorithm successfully identified the most specific hypothesis for the crop growth dataset as ['Black', 'High', '?', '?'].

2. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

Problem 1: EnjoySport Prediction

1. Question

For a given set of training examples stored in a CSV file, implement the Candidate-Elimination algorithm in Python to find all hypotheses consistent with the EnjoySport examples.

2. Aim

To implement the Candidate-Elimination algorithm and identify the version space for predicting whether a person enjoys sports.

3. Algorithm

Read the training data from a CSV file.

Generate all candidate hypotheses from the attribute values.

Compare each hypothesis with every training example.

Reject hypotheses that disagree with any positive or negative example.

Display all remaining consistent hypotheses.

4. Code

First, create a file named training.csv with the following data:

```
Sky,Temperature,Wind,EnjoySport
Sunny,Warm,Strong,Yes
Sunny,Warm,Weak,Yes
Rainy,Cold,Strong,No
Sunny,Cold,Strong,No
```

Save this Python program as candidate1.py.

```
import csvfrom itertools
import productwith open("training.csv", "r", newline="")
```

```
as file:
rows = list(csv.reader(file))
data = rows[1:]
attributes = rows[0][:-1]
domains = [    sorted(set(row[i] for row in data))
for i in range(len(attributes))]
hypotheses = product(    *[domain + ["?"]
for domain in domains])version_space = []
for h in hypotheses:
valid = True
for row in data:
match = all(h[i] == "?" or h[i] == row[i]
for i in range(len(attributes))       )
if row[-1] == "Yes" and not match:
valid = False
break
if row[-1] == "No" and match:
valid = False
```

5. Input

The program reads the four examples from training.csv.

6. Output

```
Consistent Hypotheses:
['Sunny', 'Warm', '?']
Total Hypotheses: 1
```

7. Result

Thus, the Candidate-Elimination algorithm identified the hypothesis ['Sunny', 'Warm', '?'] as consistent with all training examples.

Problem 2: Student Placement Prediction

1. Question

Implement the Candidate-Elimination algorithm using a CSV file to find all hypotheses consistent with student placement training examples.

2. Aim

To determine the attribute combinations consistent with whether a student gets placed.

3. Algorithm

Read the student placement dataset from a CSV file.

Generate possible hypotheses.

Compare each hypothesis against positive and negative examples.

Remove all inconsistent hypotheses.

Display the version space.

3. Code

Use the same Python code provided in Problem 1,

saved as candidate2.py.

Create training.csv with the following

data:

Marks,Communication,Internship,Placed

High,Good,Yes,Yes
High,Good,No,Yes
Low,Poor,No,No
Medium,Good,Yes,No
4.

5. Input

Marks	Communication	Internship	Placed
High	Good	Yes	Yes
High	Good	No	Yes
Low	Poor	No	No
Medium	Good	Yes	No

6. Output

Consistent Hypotheses:
['High', 'Good', '?']
Total Hypotheses: 1

7. Result

Thus, the Candidate-Elimination algorithm identified ['High', 'Good', '?'] as the hypothesis consistent with all student placement examples.

Problem 3: Weather-Based Picnic Prediction

1. Question

Implement the Candidate-Elimination algorithm using a CSV file to find hypotheses consistent with weather conditions for deciding whether a picnic is suitable.

2. Aim

To identify the weather conditions consistent with the picnic training examples.

3. Algorithm

Read the picnic dataset from a CSV file.

Generate possible hypotheses from the attribute values.

Test each hypothesis against every example.

Reject hypotheses that disagree with any training label

Display all remaining hypotheses.

4. Code

Use the same Python code provided in Problem 1,

saved as candidate3.py.

Create training.csv with the following data:

```
Weather,Temperature,Humidity,Picnic
Sunny,Warm,Low,Yes
Sunny,Mild,Low,Yes
Rainy,Cold,High,No
Cloudy,Cold,High,No
```

5. Input

Weather	Temperature	Humidity	Picnic
Sunny	Warm	Low	Yes
Sunny	Mild	Low	Yes
Rainy	Cold	High	No
Cloudy	Cold	High	No

6. Output

```
Consistent Hypotheses:
['Sunny', '?', 'Low']
Total Hypotheses: 1
```

7. Result

Thus, the algorithm identified ['Sunny', '?', 'Low'] as the hypothesis consistent with the picnic training data.

Problem 4: Laptop Purchase Preference

1. Question

Implement the Candidate-Elimination algorithm using a CSV file to identify all hypotheses consistent with a customer's laptop purchase preferences.

2. Aim

To determine the laptop characteristics consistent with the customer's purchase decisions.

3. Algorithm

Read the laptop dataset from a CSV file.

Generate possible hypotheses using the attribute value

Compare each hypothesis with all positive and negative examples.

Remove hypotheses that are inconsistent with the data.

Print the remaining hypotheses.

4. Code

Use the same Python code provided in Problem 1,

saved as candidate4.py.

Create training.csv with the following data:

```
Brand,RAM,Storage,Purchase
Dell,16GB,512GB,Yes
HP,16GB,512GB,Yes
Lenovo,8GB,256GB,No
Apple,8GB,256GB,No
```

5. Input

Brand	RAM	Storage	Purchase
Dell	16GB	512GB	Yes
HP	16GB	512GB	Yes
Lenovo	8GB	256GB	No
Apple	8GB	256GB	No

6. Output

```
Consistent Hypotheses:
['?', '16GB', '512GB']
Total Hypotheses: 1
```

7. Result

Thus, the Candidate-Elimination algorithm identified ['?', '16GB', '512GB'] as the hypothesis consistent with all laptop purchase examples.

Problem 5: Crop Growth Prediction

1. Question

Implement the Candidate-Elimination algorithm using a CSV file to identify hypotheses consistent with crop growth conditions.

2. Aim

To determine the soil, rainfall, and temperature characteristics consistent with successful crop growth.

3. Algorithm

Read the crop growth data from a CSV file.

Generate all possible hypotheses from the attributes.

Check every hypothesis against all training examples.

Reject hypotheses that disagree with any positive or negative example.

display the final version space.

4. Code

Use the same Python code provided in Problem 1,

saved as candidate5.py.

Create training.csv with the following data:

```
Soil,Rainfall,Temperature,Growth
Black,High,Warm,Yes
Black,High,Cold,Yes
Red,Low,Cold,No
Red,Low,Warm,No
```

5. Input

Soil	Rainfall	Temperature	Growth
Black	High	Warm	Yes
Black	High	Cold	Yes
Red	Low	Cold	No
Red	Low	Warm	No

6. Output

Consistent Hypotheses:
['Black', 'High', '?']
Total Hypotheses: 1

7. Result

Thus, the Candidate-Elimination algorithm identified ['Black', 'High', '?'] as the hypothesis consistent with all crop growth examples.

3.Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample

Experiment 1: Play Tennis Prediction

Question: Demonstrate the working of the decision tree-based ID3 algorithm using weather data to decide whether to play tennis. Build a decision tree and classify a new sample.

Aim: To implement the ID3 algorithm and predict whether tennis can be played based on weather conditions.

Algorithm:

Create a dataset containing weather conditions and the decision to play tennis.

Calculate the entropy of the target class.

Calculate information gain for each attribute.

Select the attribute with the highest information gain.

Repeat the splitting process to build the decision tree

Classify a new sample using the trained tree.Code:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier,
export_textdata = {"Outlook":
["Sunny","Sunny","Overcast","Rain","Rain","Rain","Overcast","Sunny","Sunny","Rain","Su
nny","Overcast","Overcast","Rain"],"Temperature":
["Hot","Hot","Hot","Mild","Cool","Cool","Cool","Mild","Cool","Mild","Mild","Mild","Hot","Mi
ld"],"Humidity":
["High","High","High","High","Normal","Normal","Normal","High","Normal","Normal","Nor
mal","High","Normal","High"],"Wind":
["Weak","Strong","Weak","Weak","Weak","Strong","Strong","Weak","Weak","Weak","St
rong","Strong","Weak","Strong"],"Play":
["No","No","Yes","Yes","Yes","No","Yes","No","Yes","Yes"Yes","Yes","Yes","No"]}
df = pd.DataFrame
(data)X = pd.get_dummies(df.drop("Play", axis=1))
y = df["Play"]model = DecisionTreeClassifier(criterion="entropy",
```

```
random_state=0)model.fit(X, y)
print("Decision Tree:")
print(export_text(model, feature_names=list(X.columns)))
sample = pd.DataFrame
([{"Outlook":"Sunny","Temperature":"Cool",
```

Input dataset: The 14 weather records in the code are the training data.

New sample:

Outlook = Sunny
Temperature = Cool
Humidity = High
Wind = Strong

Output:

Decision Tree:
[Tree structure printed by the program]

New Sample: {'Outlook': 'Sunny', 'Temperature': 'Cool',
'Humidity': 'High', 'Wind': 'Strong'}
Prediction: No

Result: The decision tree was trained using entropy, and the new weather sample was classified as No, meaning tennis is not recommended for that sample.

Experiment 2: Student Pass/Fail Prediction

Question: Demonstrate the ID3 algorithm using student academic data and classify a new student as Pass or Fail.

Aim: To build a decision tree using study habits, attendance, and assignment completion.

Algorithm:

Create a dataset containing student details and results.

Calculate entropy and information gain.

Choose the attribute with the highest information gain.

Build the decision tree using the selected attributes.

Give a new student's details as input.

Predict whether the student will Pass or Fail.

Code:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier,
export_textdata =
{   "Study": ["Good", "Good", "Poor", "Poor", "Good",          "Poor", "Good", "Poor"],
"Attendance": ["High", "High", "Low", "Low", "Low",High", "High", "Low"],   "Assignment":
["Complete", "Complete", "Incomplete","Complete", "Complete", "Incomplete",
"Incomplete", "Incomplete"],"Result":
["Pass","Pass","Fail","Fail","Pass","Fail","Pass","Fail"]}
df = pd.DataFrame
(data)X = pd.get_dummies(df.drop("Result", axis=1))
y = df["Result"]model = DecisionTreeClassifier(criterion="entropy",
random_state=0)model.fit(X, y)print("Decision Tree:")
print(export_text(model, feature_names=list(X.columns)))
sample =
pd.DataFrame([{"Study":"Good","Attendance":"High","Assignment":"Complete"}])
sample_encoded = pd.get_dummies(sample).reindex(   columns=X.columns,
```

```
fill_value=0)
```

Input dataset: Eight student records are included in the code.

New sample:

Study = Good
Attendance = High
Assignment = Complete

Output:

```
Decision Tree:
[Tree structure printed by the program]

New Student: {'Study': 'Good', 'Attendance': 'High',
              'Assignment': 'Complete'}
Predicted Result: Pass
```

Result: The ID3-style decision tree classified the new student as Pass.

Experiment 3: Loan Approval Prediction

Question: Implement the ID3 decision tree algorithm using a loan approval dataset and predict whether a new applicant's loan will be approved.

Aim: To classify a loan application based on income, credit history, and employment status.

Algorithm:

Prepare the loan applicant dataset.

Calculate the entropy of the loan approval target.

Find information gain for each attribute.

Select the best attribute and build the decision tree.

Encode the new applicant's details.

Predict loan approval.

Code:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier,
export_textdata =
{"Income": ["High", "High", "Low", "Low", "Medium",          "High", "Medium", "Low"],
"Credit": ["Good", "Good", "Poor", "Poor", "Good",          "Poor", "Good", "Poor"],
"Employment": ["Stable", "Unstable", "Stable",
"Unstable",Stable", "Stable", "Unstable", "Stable"],
"Loan": ["Yes", "Yes", "No", "No", "Yes",
"No","Yes","No"]}
df = pd.DataFrame(data)X = pd.get_dummies(df.drop("Loan", axis=1))
y = df["Loan"]model = DecisionTreeClassifier(criterion="entropy",
random_state=0)model.fit(X, y)
```

```
print("Decision Tree:")
print(export_text(model, feature_names=list(X.columns)))
sample = pd.DataFrame([{   "Income": "High",
"Credit":"Good","Employment":"Stable"}])
sample_encoded = pd.get_dummies(sample).reindex(   columns=X.columns,
fill_value=0)
print("New Applicant:",
sample.to_dict(orient="records")[0])
```

Input dataset: Eight loan application records are included in the code.

New sample:

Income = High
Credit = Good
Employment = Stable

Output:

Decision Tree:
[Tree structure printed by the program]

New Applicant: {'Income': 'High', 'Credit': 'Good',
'Employment': 'Stable'}
Loan Decision: Yes

Result: The decision tree predicted Yes, indicating that the sample loan application is classified as approved by the model. This is a lab demonstration, not a real financial decision.

Experiment 4: Fruit Classification

Question: Demonstrate the ID3 algorithm using fruit data and classify a new fruit based on its color, shape, and size.

Aim: To build a decision tree that identifies the type of fruit from its features.

Algorithm:

Create a dataset containing fruit characteristics and fruit names.

Calculate entropy and information gain for the attributes.

Select the attribute with the highest information gain.

Build the decision tree from the training dataset.

Enter the features of a new fruit.

Predict its fruit category using the trained tree.

Code:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier,
export_textdata =
{ "Color": ["Red", "Red", "Yellow", "Yellow","Green", "Green", "Red", "Yellow"],   "Shape":
["Round", "Round", "Long", "Long",
"Round", "Long", "Round", "Round"],
"Size": ["Medium", "Large", "Medium", "Large",
"Small", "Medium", "Small", "Small"],
"Fruit": ["Apple", "Apple", "Banana", "Banana",
"Apple","Banana","Apple","Apple"]}
df = pd.DataFrame(data)X = pd.get_dummies(df.drop("Fruit", axis=1))y = df["Fruit"]
model = DecisionTreeClassifier(criterion="entropy", random_state=0)model.fit(X, y)
print("Decision Tree:")
print(export_text(model, feature_names=list(X.columns)))
sample = pd.DataFrame
([{"Color":"Red","Shape":"Round","Size":"Medium"}])
```

```
sample_encoded = pd.get_dummies(sample).reindex(    columns=X.columns,
fill_value=0)
print("New Fruit:", sample.to_dict(orient="records")[0])
```

Input dataset: Eight fruit records are included in the code.

New sample:

Color = Red
Shape = Round
Size = Medium

Output:

Decision Tree:
[Tree structure printed by the program]

New Fruit: {'Color': 'Red', 'Shape': 'Round', 'Size': 'Medium'}
Predicted Fruit: Apple

Result: The decision tree classified the new fruit as Apple.

Experiment 5: Crop Growth Prediction

Question: Demonstrate the ID3 decision tree algorithm using agricultural data to predict crop growth based on soil type, water availability, and sunlight.

Aim: To develop a decision tree that classifies crop growth as Good or Poor.

Algorithm:

Create a dataset containing soil, water, sunlight, and crop growth.

Calculate the entropy of the target class.

Calculate information gain for each input attribute.

Select the best attribute for splitting the data.

Construct the decision tree.

Predict the growth category for a new sample.

Code:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier,
export_textdata =
{   "Soil": ["Black", "Black", "Red", "Red",
"Black", "Red", "Black", "Red"],
"Water": ["Adequate", "Adequate", "Low", "High",
"High", "Low", "Low", "High"],
"Sunlight": ["High", "Low", "High", "High",
"High", "Low", "Low", "Low"],
"Growth": ["Good", "Good", "Poor", "Poor",
"Good","Poor","Good","Poor"]}
df = pd.DataFrame(data)
X = pd.get_dummies(df.drop("Growth", axis=1))
y = df["Growth"]
```

```
model = DecisionTreeClassifier(criterion="entropy", random_state=0)model.fit(X,
y)print("Decision Tree:")
print(export_text(model, feature_names=list(X.columns)))
sample = pd.DataFrame([{    "Soil": "Black",
"Water":"Adequate","Sunlight":"High"}])
sample_encoded = pd.get_dummies(sample).reindex(    columns=X.columns,
fill_value=0)
print("New Crop Sample:", sample.to_dict(orient="records")[0])
```

New sample:

Soil = Black
Water = Adequate
Sunlight = High

Output:

Decision Tree:
[Tree structure printed by the program]

New Crop Sample: {'Soil': 'Black', 'Water': 'Adequate',
'Sunlight': 'High'}
Predicted Growth: Good

Result: The decision tree classified the new crop sample as having Good growth.